

MODUL DEP DEPOYMENT

Nicolas Dumermuth

Inhaltsverzeichnis

Inhaltsverzeichnis.....	1
Modulleitfaden	2
1. Verortung im Curriculum.....	2
2. Umfang und Lernstunden	2
3. Kompetenzen	3
3.1 Fachliche Kompetenzen.....	3
3.2 Überfachliche Kompetenzen	3
4. Vorkenntnisse	4
5. Hinweise zum Selbststudium.....	5
6. Bezug zu RLP Handlungskompetenzen	6
7. Semesterplan.....	9
8. Arbeits- und Leistungsnachweise	10
9. Modulspezifische Hinweise	11
9.1 Lernumgebung smartlearn.online.....	11

Modulleitfaden

1. Verortung im Curriculum

Das Modul DEP (Deployment) vermittelt die professionelle Auslieferung und den Betrieb von Applikationen. Die Studierenden lernen, Anwendungen containerisiert, automatisiert und reproduzierbar in verschiedene Umgebungen auszuliefern. Das Modul deckt den gesamten DevOps-Lifecycle ab: von Docker-Containerisierung über CI/CD-Pipelines mit GitHub Actions bis hin zu Cloud-Deployment und Kubernetes. Es baut auf den Programmier- und Webentwicklungsmodulen (PRG, WEB) auf und ergänzt diese um die operative Seite der Softwarebereitstellung.

Curriculum 2026

	1. Semester	2. Semester	3. Semester	4. Semester	5. Semester mit Schwerpunkten	6. Semester
Computer Science	Programming Algorithm & Datastructures - PAD Network I - NB I System Administration I - SA I	Object Oriented Programming - OOP Network Basics II - NB II System Administration II - SA II	Python Scripting Automation - PSA Cyber Security I - CYS I Data Management I - DB I Microsoft System Administration I - MSA I	Programming I - PRG I Cyber Security II - CYS II Internet of Things - IOT Webengineering I - WEB I	Application Programming II - PRG II Deployment - DEP Webengineering II - WEB II Machine Learning - ML	Platformengineering Scripting II - SCR II Cloud Computing II - CC II Microsoft System Administration II - MSA II Monitoring - Mon
Praktika					Practical Work A Programming III - PRG III Practical Work B Agile Development Process - ADP	
Social & Economics	Selfmanagement I - SELF	Digital Ethics - DE	Digital Transformation I - DTR I Leadership I - LDS I	Digital Transformation II - DTR II Leadership II - LDS II		
Languages	English A) B - ENG A) B	English A) B - ENG A) B				
Natural Sciences and Mathematics	Mathematics I - MATH I	Mathematics II - MATH II				
Basics			Project Management Methodology I - PMM I	Project Management Methodology II - PMM II IT Service Management - ISM		

Diplomarbeit und Diplomprüfungen

2. Umfang und Lernstunden

LERNFORM	ANZAHL LEKTIONEN	BEMERKUNGEN
KONTAKTSTUDIUM	28	Seminare, Workshops
ANGELEITETES SELBSTSTUDIUM	12	Projektarbeit
INDIVIDUELLES SELBSTSTUDIUM	40	Individuelle Vertiefung
TOTAL LEKTIONEN	80	

3. Kompetenzen

3.1 Fachliche Kompetenzen

- › **Deployment-Grundlagen**
 - › Fundiertes Verständnis von Deployment-Konzepten, Environments, Build-Prozessen und der 12-Factor App
 - › Analyse von Deployment-Strategien (Big Bang, Rolling Update, Blue/Green, Canary) und deren Einsatzszenarien
 - › Bewertung von Infrastructure as Code, CI/CD-Pipelines und der 12-Factor App Methodik
- › **Docker und Containerisierung**
 - › Erstellung von Dockerfiles mit Best Practices (Multi-Stage Builds, Layer Caching, .dockerignore)
 - › Aufbau von Multi-Service-Architekturen mit Docker Compose (Netzwerke, Volumes, Environment-Variablen)
 - › Containerisierung eigener Applikationen mit reproduzierbaren Builds und Port-Mapping
- › **CI/CD und Cloud-Deployment**
 - › Erstellung automatisierter CI/CD-Pipelines mit GitHub Actions (Build, Test, Deploy)
 - › Deployment auf Cloud-Plattformen (Railway, Render, Fly.io, Vercel)

3.2 Überfachliche Kompetenzen

- › **Methodenkompetenzen**
 - › Systematische Planung und Dokumentation von Deployment-Architekturen und CI/CD-Pipelines
 - › Analyse und Behebung komplexer Deployment-Probleme in containerisierten Umgebungen
 - › Prozessorientiertes Denken bei der Gestaltung von Deployment-Pipelines und Release-Prozessen
 - › Effektive Nutzung von Dokumentation und Best-Practice-Guides
 - › Selbstgesteuertes Lernen durch praktische Labs und eigenständige Projekte
- › **Sozialkompetenzen**
 - › Kommunikationsfähigkeit beim Präsentieren von Deployment-Entscheiden und Architekturkonzepten (Screencast, Dokumentation)
 - › Teamfähigkeit in Projektarbeiten mit Peer-Review und gemeinsamem Troubleshooting
 - › Konstruktive Feedback-Kultur bei Architekturentscheidungen
- › **Selbstkompetenzen**
 - › Eigenverantwortung beim selbstständigen Durcharbeiten der smartlearn-Aufträge
 - › Flexibilität im Umgang mit neuen Deployment-Technologien und Cloud-Plattformen
 - › Ethisches Handeln mit Bewusstsein für Secrets-Management, sichere Konfiguration und verantwortungsvollen Umgang mit KI-Tools im Entwicklungsprozess

4. Vorkenntnisse

Das Modul baut auf den Grundlagen aus PRG1/2/3 und WEB1/2 auf. Die Studierenden sollten über solide Grundkenntnisse in einer Programmiersprache (z.B. Node.js, Python oder .NET) verfügen sowie grundlegende Erfahrung mit Git und GitHub mitbringen. Kenntnisse in der Webentwicklung (HTML, CSS, JavaScript) aus WEB1/WEB2 sind ebenso erforderlich. Erfahrung mit der Kommandozeile und einem Code-Editor (VS Code) ist hilfreich.

Das Modul ergänzt die Entwicklungsmodule um die operative Perspektive: Wie wird Code zuverlässig und automatisiert in Produktion gebracht? Dieser Brückenschlag zwischen Entwicklung und Betrieb (DevOps) ist eine Schlüsselkompetenz moderner Softwareentwicklung.

Spezifische Docker- oder DevOps-Vorkenntnisse sind nicht erforderlich – alle Deployment-Konzepte werden im Modul von Grund auf vermittelt. Docker Desktop muss vor dem zweiten Unterrichtstag installiert und funktionsfähig sein. Sollten Wissenslücken in den Vorkenntnissen bestehen, sind diese in Eigenregie aufzuarbeiten.

5. Hinweise zum Selbststudium

Das Modul kombiniert strukturierte Präsenzphasen mit intensivem Selbststudium über die smartlearn-Plattform. Die Unterlagen sind so konzipiert, dass sie sowohl im Präsenzunterricht als auch im Selbststudium funktionieren und vollständig studiert werden sollten. Bei Literaturangaben oder weiterführenden Links können Sie individuell entscheiden, ob die Inhalte bereits bekannt sind oder noch vertieft werden müssen.

Im Präsenzunterricht demonstriert der Dozierende Deployment-Konzepte und deren praktische Umsetzung mit Docker, Docker Compose, GitHub Actions und Cloud-Plattformen. Die Studierenden arbeiten auf ihrem eigenen Rechner mit Docker Desktop und VS Code. Dabei entsteht ein gemeinsamer Lernprozess mit Troubleshooting und Diskussionen über Best Practices. Das angeleitete Selbststudium vertieft diese Erfahrungen durch praktische Deployment-Challenges zu Docker Compose, CI/CD, Cloud-Deployment und Kubernetes.

Die Aufträge auf smartlearn werden in Eigenverantwortung abgeschlossen. Die Deployment-Challenges können einzeln oder im Team und mit Hilfe von KI gelöst werden. KI-Tools dürfen verwendet werden, müssen aber im README deklariert werden. Der Output steht im Fokus – wichtig ist, dass Sie die Lösungen immer verstehen und nachvollziehen können.

Ein wichtiger Aspekt ist die Dokumentation Ihrer Deployment-Entscheide. Halten Sie bei jedem Auftrag fest, warum Sie bestimmte Technologien, Konfigurationen und Strategien gewählt haben. Die Erstellung von Screencasts (Video-Dokumentationen) ist ein zentraler Bestandteil der Leistungsnachweise – üben Sie dies frühzeitig. Reflexionsfragen wie “Warum habe ich diese Deployment-Strategie gewählt?” oder “Wie würde ich dieses Setup für eine Produktionsumgebung absichern?” helfen dabei, ein tiefes Verständnis zu entwickeln.

6. Bezug zu RLP Handlungskompetenzen

Die Grundlage für die nachfolgenden Inhalte bildet der Rahmenlehrplan HF Informatik. Er legt die zentralen Bildungsziele, Kompetenzen und Schwerpunkte der Ausbildung an höheren Fachschulen fest und dient damit als verbindlicher Bezugspunkt für die Gestaltung des Unterrichts.

<https://www.becc.admin.ch/becc/public/bvz/beruf/show/321>

Kompetenz (RLP-Ref.)	Beschreibung	Niveau (1–4)	Konkretisierung im Modul
B4.2	ICT-Problemstellungen unter Berücksichtigung vernetzten Denkens, Entwicklung neuer ICT-Lösungen und Anwendung aktueller Technologien identifizieren, analysieren und lösen	3	Studierende analysieren komplexe Deployment-Problemstellungen systematisch. Sie evaluieren Deployment-Strategien (Rolling Update, Blue/Green, Canary) und wählen geeignete Ansätze für verschiedene Szenarien. Praktische Probleme lösen sie durch Containerisierung mit Docker und entwickeln automatisierte Lösungen mit CI/CD-Pipelines (GitHub Actions).
B4.6	Aktuelle technologiebasierte Entwicklungswerkzeuge einsetzen	3	Praktischer Einsatz moderner Deployment-Werkzeuge: Containerisierung mit Docker und Docker Compose, CI/CD-Automatisierung mit GitHub Actions, Cloud-Deployment auf Plattformen wie Railway/Render/Fly.io, Container-Orchestrierung mit Kubernetes. Konfiguration und Betrieb dieser Tools zur automatisierten Auslieferung von Applikationen.
B5.1	ICT-Projekte oder Vorhaben eigenständig bis zur Ausführungsreife 3 planen und steuern	3	Eigenständige Planung eines produktionsreifen Deployments einer eigenen Applikation. Strukturierung in Teilschritte (Containerisierung, Multi-Service-Architektur, CI/CD-Pipeline, Cloud-Deployment) und systematische Steuerung bis zur funktionsfähigen Auslieferung im Vertiefungsprojekt (LB2).

B8.1	Das ICT-Qualitätsmanagement-System definieren, dokumentieren, umsetzen und überwachen	3	Definition und Umsetzung von Qualitätssicherungsmassnahmen im Deployment-Prozess: Automatisierte Tests in CI/CD-Pipelines, Health Checks (Liveness/Readiness Probes), reproduzierbare Builds mit Lock-Dateien und Multi-Stage Dockerfiles. Überwachung der Deployment-Qualität durch strukturierte Code-Reviews und Dokumentation.
B10.1	Die Architektur der Software bestimmen und die Entwicklung unter Berücksichtigung von Betrieb und Wartung planen, realisieren und dokumentieren	3	Bestimmung der Softwarearchitektur unter Berücksichtigung von Deployment-Anforderungen: Multi-Service-Architekturen mit Docker Compose, Trennung von Build/Release/Run gemäss 12-Factor App. Planung der Applikationsstruktur für containerisiertes Deployment mit Fokus auf Wartbarkeit, Skalierbarkeit und Betrieb.
B9.1	ICT-Sicherheitskonzepte zur Gewährleistung der ICT-Sicherheit wie Datenschutz, Datensicherheit und Verfügbarkeit einhalten, umsetzen und unterhalten	3	Praktische Umsetzung von Sicherheitskonzepten im Deployment: Secrets-Management mit Environment-Variablen (keine Credentials im Code), .env-Dateien und GitHub Secrets. Sichere Docker-Konfigurationen (.dockerignore, nicht als Root laufen). Analyse von Security-Best-Practices für CI/CD-Pipelines und Cloud-Deployments.
B11.4	Entwicklungsprojekte aufgrund der Analyseergebnisse und des gewählten Vorgehens planen und leiten	3	Planung und Leitung des Vertiefungsprojekts (LB2): Eigenständige Wahl eines Deployment-Themas, Erstellung eines Architekturkonzepts, Umsetzung eines produktionsreifen Deployments mit CI/CD-Pipeline. Dokumentation der Entscheide und Erstellung eines Screencasts zur Präsentation der Ergebnisse.
B11.6	Testkonzepte und Testspezifikation erstellen, Tests implementieren und auswerten sowie notwendige Massnahmen umsetzen	3	Integration von automatisierten Tests in CI/CD-Pipelines mit GitHub Actions: Linting, Unit-Tests und Build-Validierung als Pipeline-Steps. Erstellung von Testspezifikationen für Deployment-Prozesse (Health Checks, Smoke Tests nach Deployment). Auswertung von Pipeline-Ergebnissen und Ableitung von Verbesserungsmassnahmen.

B13.4	Anforderungen aus dem Service-Management analysieren, entwickeln und integrieren	3	Analyse von Service-Anforderungen (Verfügbarkeit, Performance, Skalierbarkeit) für Deployment-Umgebungen. Implementierung von Health-Checks und Monitoring-Endpunkten in containerisierten Applikationen. Integration von Load-Balancing und Service-Discovery in Kubernetes (Services, Ingress, NodePort).
B14.1	Technische und organisatorische Maßnahmen planen und für die Einführung von Software bzw. Releases ausarbeiten	3	Schrittweise Einführung von Applikationen über automatisierte Pipelines. Erstellung von Docker-Compose- und Kubernetes-Manifesten (YAML), Definition von Deployment-Strategien (Rolling Update, Blue/Green), Konfiguration von Health-Probes (Liveness, Readiness). Planung und Durchführung von Release-Prozessen auf Cloud-Plattformen.
B15.1	Neue und bereits bestehende ICT-Systeme bezüglich Energieeffizienz und Umweltverträglichkeit evaluieren	2	Evaluation von Deployment-Architekturen unter Nachhaltigkeitsaspekten. Vergleich des Ressourcenverbrauchs verschiedener Deployment-Ansätze (lokale Container vs. Cloud), Analyse von Cloud-Kostenmodellen (Free Tier, Pay-as-you-go), Bewertung von Multi-Stage Builds und Image-Optimierung zur Reduktion von Speicher- und Bandbreitenverbrauch.

7. Semesterplan

KW	Datum	Inhalt/ Thema	Anzahl L.	Lernform	Sozialform	Lernort	Abgaben
KW 17	25.04.2026	Deployment-Grundlagen: Environments, Builds, 12-Factor App Docker Grundlagen, Dockerfile, Container	K/AS	K/AS	Präsenz	Präsenz	
KW 18	02.05.2026	Deployment-Tools und Challenge-Start Docker Compose, CI/CD, Cloud, K8s	4L	K/AS	EA	Präsenz	
KW 19	09.05.2026	Selbständige Arbeit Challenges Fernunterricht, eigenständiges Arbeiten Coaching via Teams	4L	IS	EA	Fernunterricht	Abgabe LB1
KW 21	23.05.2026	A4 : Infrastructure as Code: Besprechung LB1, Kick-off Vertiefungsprojekt (LB2)	4L	K/AS	EA/GA	Präsenz	
KW 22	30.05.2026	Projektarbeit und Coaching (Cloud Grundlagen, Private Cloud, IaC) Einführung PaaS-Lösungen	4L	K/AS	EA/GA	Präsenz	Abgabe LB2
KW 23	06.06.2026	Abschluss: Besprechung LB2-Ergebnisse Refresher WEB2	4L	K/AS	EA/GA	Präsenz	

8. Arbeits- und Leistungsnachweise

Bereich	Kriterien	Bewertung/ Gewichtung
LB1 Deployment-Aufträge Einzelarbeit	Die Studierenden wählen und lösen mindestens 3 von 5 Deployment-Challenges (Docker Compose, CI/CD, Cloud, Kubernetes, Eigene Idee). Pro Auftrag werden max. 10 Punkte vergeben (Umsetzung 4, Dokumentation 3, Screencast 3). Die Abgabe erfolgt per Teams mit Git-Repository-Link, Dokumentation und Screencast. KI-Tools sind erlaubt und müssen im README deklariert werden.	Note (40%)
LB2: Vertiefungsprojekt Einzel- oder Partnerarbeit	Vertiefung eines Wahlthemas aus den Deployment-Aufträgen. Die Studierenden deployen eine eigene Applikation produktionsreif mit CI/CD-Pipeline, erstellen eine vollständige Dokumentation (README mit Setup-Anleitung, Architektur-Diagramm, Entscheide, Learnings) und einen Screencast (5–10 Min.). Max. 30 Punkte gemäss Bewertungsraster (Architektur 6, Umsetzung 8, Docker Best Practices 4, Dokumentation 6, Screencast 6). Einzel- oder Partnerarbeit möglich.	Note (60%)

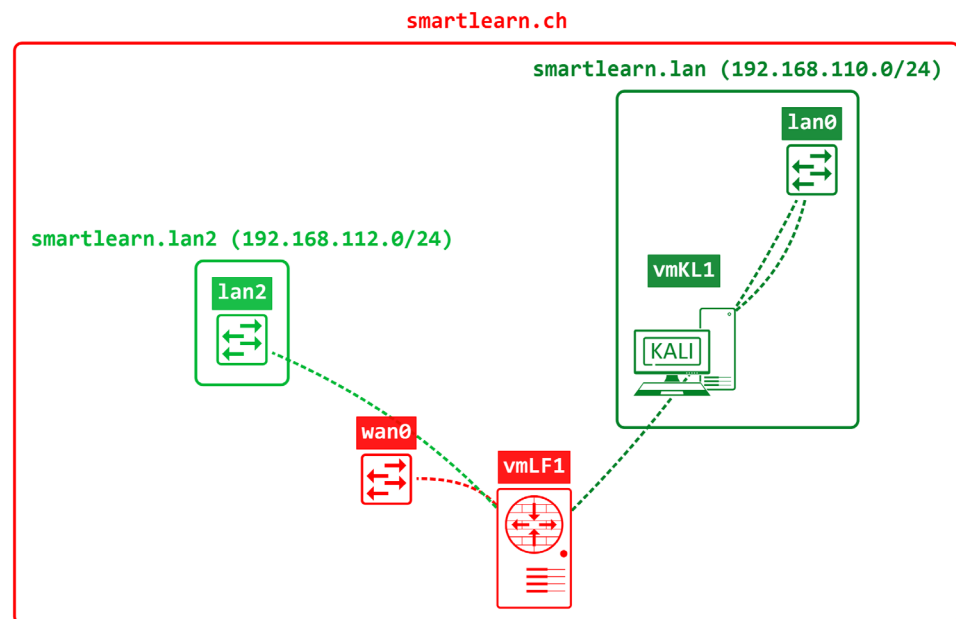
9. Modulspezifische Hinweise

9.1 Lernumgebung smartlearn.online

Das Modul basiert primär auf Docker Desktop und GitHub Actions. Die Studierenden arbeiten auf ihrem eigenen Rechner. Folgende Tools werden eingesetzt:

- › Docker Desktop: Container-Plattform inkl. Docker Compose V2 für Containerisierung und Multi-Service-Apps
- › Git und GitHub: Versionsverwaltung, Code-Hosting und CI/CD mit GitHub Actions
- › VS Code: Empfohlene IDE mit Docker- und YAML-Extensions

Als Alternative stehen in smartlearn die vmKL1 (Kali-Desktop) und die vmLR1 (Alpine Linux) zum Lösen der Aufgaben bereit.



Alle Unterlagen werden auf smartlearn bereitgestellt und während des Kurses laufend nachgeführt. Die Plattform umfasst:

- › Aufträge & praktische Übungen (Challenges)
- › Zusätzliche Dokumente & Beispiele
- › Erklärvideos und Tutorials
- › Zusätzliche Programme und Hilfsmittel
- › Vorlagen und Lösungen